Red ASM Security

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

SP-108-RedASM

CS 4850 – Computer Science Senior Project

Spring 2026

Professor Perry

February 8th, 2026

Adam Llado | Kimani Gordon | Alec Sundby | Austin Abeln

1.0 Introduction 3
1.1 Overview 3
1.2 Project Goals 3
1.3 Definitions and Acronyms 3
1.4 Assumptions 4
2.0 Design Constraints 4
2.1 Environment 4
2.2 User Characteristics 4
2.3 System 4
3.0 Functional Requirements 4
3.1 Asset Discovery 4
3.2 Vulnerability Detection 5
3.3 Asset Classification 5
3.4 Logging and Data Management 5
3.5 SIEM Integration 5
3.6 SOAR Automation 5
3.7 Alerting and Notification 5
4.0 Non-Functional Requirements 6
4.1 Security 6
4.2 Capacity 6
4.3 Usability 6
4.4 Other 6
5.0 External Interface Requirements 6
5.1 User Interface Requirements 6
5.2 Hardware Interface Requirements 6
5.3 Software Interface Requirements 7
5.4 Communication Interface Requirements 7

1.0 Introduction

1.1 Overview

Red ASM Security is a cyber-attack surface management (ASM) system designed to simulate a small-scale security operations center (SOC) environment. The system provides continuous asset discovery, vulnerability identification, risk classification, alerting, and automated response for a monitored target virtual machine (VM). The project emulates both red team discovery techniques and blue team defensive operations using open-source security tooling.

The system consists of two primary components:
- A **Target VM**, representing an enterprise system with intentional and unintentional exposure.
- A **Manager VM**, acting as a centralized SOC platform that performs SIEM and SOAR functions.

Red ASM Security demonstrates how software and configuration drift lead to the accumulation of shadow IT and how automated security tooling can be used to discover, classify, and respond to these risks.

1.2 Project Goals

The Objective of Red ASM Security:

- To simulate a realistic attack surface management workflow.
- To demonstrate continuous asset discovery from an attacker's perspective.
- To classify discovered assets as business-required, known-risk, or shadow IT.
- To assess vulnerabilities using CVSS and CVE data.
- To implement automated alerting and response workflows.
- To showcase integration between ASM tools, SIEM, SOAR, and notification platforms.
- To analyze the progression of software and configuration drift over time.

1.3 Definitions and Acronyms

- **ASM**: Attack Surface Management
- **SOC**: Security Operations Center
- **SIEM**: Security Information and Event Management
- **SOAR**: Security Orchestration, Automation, and Response
- **Shadow IT**: Unmanaged or forgotten IT assets that create security risk
- **CVSS**: Common Vulnerability Scoring System
- **CVE**: Common Vulnerabilities and Exposures
- **VM**: Virtual Machine

1.4 Assumptions

- The system operates in a controlled lab or academic environment.
- Open-source tools are sufficient for asset discovery and monitoring.
- Network connectivity exists between the Manager VM and Target VM.
- Users have basic cybersecurity and Linux system knowledge.
- Asset discovery scans are authorized and ethical.

2.0 Design Constraints

2.1 Environment

- Linux-based virtual machines (e.g., Ubuntu Server)
- Local or cloud-hosted virtualized infrastructure
- Isolated network environment for testing

2.2 User Characteristics

- Primary users are cybersecurity students and analysts.
- Users are expected to understand:
    - Networking fundamentals
    - Basic system administration
    - Security concepts such as ports, services, and vulnerabilities

2.3 System

- Must rely on open-source tools (e.g., Wazuh, Shuffle, Nmap).
- Limited hardware resources due to VM-based deployment.
- SQLite used for lightweight storage instead of enterprise databases.
- Scanning frequency constrained to avoid system overload.

3.0 Functional Requirements

(list the design, graphic, and operating system requirements nd list any constraints here and assign them numbers like 2.1, 2.2, etc)

3.1 Asset Discovery

- 3.1.1 The system shall perform continuous DNS and subdomain discovery using tools such as Amass and Subfinder.
- 3.1.2 The system shall identify IP addresses and IP ranges associated with the target.
- 3.1.3 The system shall perform host and port scanning using Nmap.

- 3.1.4 The system shall discover web applications and API endpoints using Katana and HTTPX.
- 3.1.5 The system shall detect exposed cloud services using Cloud_Enum.

3.2 Vulnerability Detection

- 3.2.1 The system shall scan for exposed dashboards and misconfigurations using Nuclei templates.
- 3.2.2 The system shall associate discovered assets with known CVEs where applicable.
- 3.2.3 The system shall assign CVSS scores to discovered vulnerabilities.

3.3 Asset Classification

- 3.3.1 The system shall classify assets as:
    - Business-required
    - Known risk
    - Shadow IT
- 3.3.2 The system shall track classification changes over time.

3.4 Logging and Data Management

- 3.4.1 The system shall log the results of each scan.
- 3.4.2 The system shall maintain a historical record of discovered assets.
- 3.4.3 The system shall store vulnerability and alert data in an SQLite database.

3.5 SIEM Integration

- 3.5.1 The system shall forward relevant security events to Wazuh.
- 3.5.2 The system shall correlate asset discovery data with security events.

3.6 SOAR Automation

- 3.6.1 The system shall trigger Shuffle workflows based on predefined alert conditions.
- 3.6.2 The system shall recommend response actions for detected vulnerabilities.

3.7 Alerting and Notification

- 3.7.1 The system shall generate alerts for high and critical vulnerabilities.
- 3.7.2 The system shall send alerts to Slack.
- 3.7.3 The system shall send alerts via email.
- 3.7.4 Alerts shall include asset details, CVE information, CVSS score, and suggested remediation steps.

4.0 Non-Functional Requirements

4.1 Security

- The system shall restrict access to authorized users only.
- Sensitive configuration data shall be stored securely.
- Communications between components should use secure channels where possible.

4.2 Capacity

- The system shall support daily automated scans.
- The system shall handle multiple asset types simultaneously.
- The system shall maintain historical scan data without significant performance degradation.

4.3 Usability

- The system shall provide readable logs and alerts.
- Alerts shall be actionable and clearly structured.
- Configuration and deployment steps shall be documented.

4.4 Other

- The system shall be modular to allow tool substitution.
- The system shall be extensible for additional scanners or integrations.

5.0 External Interface Requirements

5.1 User Interface Requirements

- Command-line interfaces for scanning tools.
- Web dashboards provided by Wazuh and Shuffle.

5.2 Hardware Interface Requirements

- Virtualized hardware environment.
- Network interfaces enable VM communication.

5.3 Software Interface Requirements

- Integration with Wazuh SIEM APIs.
- Integration with Shuffle SOAR workflows.
- SQLite database interface for storage.

5.4 Communication Interface Requirements

- HTTP/HTTPS for web-based dashboards and APIs.
- Email (SMTP) for alert notifications.
- SlackWebhooks for messaging alerts.